MODULE 13 · BLOCK 3 - DATA COLLECTIONS

Tuples

The comma that makes a tuple, reading tuples like lists, what immutable really means, and packing and unpacking.

BLOCK 3 of 4 EXAM WEIGHT 25% MODULE 13 of 28

admin@sthithapragna.com

A tuple is a sequence you **cannot change**. And it is the **comma** that makes a tuple, not the brackets.

Commas make tuples

```
1  t1 = (1, 2, 3)
2  t2 = 1, 2, 3            # packing: the brackets are optional
3  t3 = (5,)               # one element needs a trailing comma
4  t4 = (5)                # no comma: this is just the int 5
5  t5 = ()                 # the empty tuple
6  print(t1 == t2, type(t3), type(t4), len(t5))
```

2 Values separated by commas become a tuple, with or without brackets.

3-4 Without the comma, brackets only group. (5) is the int 5; (5,) is a tuple.

Output: True <class 'tuple'> <class 'int'> 0

A stray comma, a stray point

```
1  y = 2 + 3 * 5,           # the trailing comma makes a tuple
2  print(y)                 # (17,)
3  print(type((2.)), type((2,)))      # float, then tuple
4  print(tuple("ab"), tuple(range(3)))  # ('a', 'b') (0, 1, 2)
```

1-2 The expression is worked out first, then the comma packs it into a one-element tuple.

3 (2.) is a float in brackets. (2,) is a tuple. One character apart.

4 tuple() converts any sequence, just as list() does.

Everything that reads a list reads a tuple

```
1  data = (1, 2, 4, 8)
2  print(data[1], data[-1])          # 2 8
3  print(data[1:3], data[-2:-1])     # (2, 4) (4,)
4  print(len(data), 4 in data)       # 4 True
5  print(data.count(2), data.index(8))   # 1 3
```

2 Indexing gives one element: an int here.

3 Slicing gives a tuple, even when only one element is left.

5 count and index are the only two tuple methods.

What you cannot do to a tuple

CODE, WITH T = (1, 2, 3)	RESULT
`t[0] = 9`	TypeError: 'tuple' object does not support item assignment
`del t[0]`	TypeError: 'tuple' object doesn't support item deletion
`t.append(4) t.sort()`	AttributeError: tuples have no such methods
`t.index(9)`	ValueError: tuple.index(x): x not in tuple
`t[3]`	IndexError: tuple index out of range
`del t`	Works: it removes the name t, not an element

THE TIEBREAKER

Changing an element: TypeError. Calling a list-only method: AttributeError.

+ and * work, because they make a new tuple

```
1  t = (1, 2)
2  t = t + (3,)              # a new tuple: (1, 2, 3)
3  print(t * 2)              # (1, 2, 3, 1, 2, 3)
4  t += (4,)                 # also a new tuple, rebound to t
5  print(t, sorted((3, 1, 2)))   # (1, 2, 3, 4) [1, 2, 3]
6  print(t + [5])            # TypeError: tuple + list
```

2-4 Nothing is changed. Each line builds a new tuple and moves the name onto it.

5 sorted() of a tuple returns a list.

6 You can only join a tuple to another tuple.

For a tuple, += always makes a new object

TUPLE

b keeps the old tuple

```
1  a = (1,)
2  b = a
3  a += (2,)
4  print(b)
```

(1,). A tuple cannot grow, so += builds a new one for a.

LIST

b sees the change

```
1  a = [1]
2  b = a
3  a += [2]
4  print(b)
```

[1, 2]. For a list, += extends the shared list in place.

ON THE EXAM

Same operator, different behaviour: immutable types rebuild, mutable types change in place.

A tuple can hold a list, and the list can still change

```
1  t = ([1, 2], "x")
2  t[0].append(3)          # fine: changes the list inside
3  print(t)                # ([1, 2, 3], 'x')
4  t[0] = [9]              # TypeError: replaces a tuple slot
```

2 The tuple still holds the same list. Only the list's contents changed.

4 Replacing the element itself is what a tuple forbids.

Several values out of one tuple

```
1  point = 3, 4              # packing
2  x, y = point              # unpacking: x is 3, y is 4
3  a, b = 1, 2
4  a, b = b, a               # the swap is pack, then unpack
5  print(x, y, a, b)         # 3 4 2 1
6  for name, score in [("Ann", 7), ("Bob", 9)]:
7      print(name, score)    # Ann 7, then Bob 9
```

2 Unpacking assigns each element to a name, in order.

4 The right side is packed into a tuple first, then unpacked into a and b.

6-7 A loop over pairs can unpack each pair into two names.

The counts must match: x, y = (1, 2, 3) raises ValueError: too many values to unpack.

Tuple versus list

TUPLE

Fixed after creation

- Written with commas; () is empty
- Only count() and index()
- Can be a dictionary key
- Suits fixed records: a point, a date

LIST

Changes as you go

- Written with []
- append, insert, sort, and more
- Cannot be a dictionary key
- Suits collections that grow

ON THE EXAM

Both support indexing, slicing, len, in, for, + and *. Only the list can be changed in place.

Tuples as keys, and comparing tuples

```
1  grid = {(0, 0): "start", (2, 3): "end"}
2  print(grid[2, 3])                    # end
3  print((1, 2) < (1, 3), (2,) > (1, 9))   # True True
```

2 Inside the square brackets, 2, 3 is packed into the tuple (2, 3).

3 Tuples compare element by element, like strings compare character by character.

What to carry into the exam

- The **comma** makes a tuple: (5,) is a tuple, (5) is an int, 1, 2 is a tuple.
- Indexing gives an element; **slicing gives a tuple**, even (4,).
- Assigning or deleting an element: **TypeError**. append or sort: **AttributeError**.
- + and * and += make a new tuple. A list inside a tuple can still change.
- Unpacking needs matching counts; tuples can be dictionary keys.

What is the output of this program?

```
1  data = (5, 10, 15, 20)
2  data = data[-3:-1]
3  data = data[-1]
4  print(data)
```

A (15,)

B (10, 15)

C The code raises a TypeError

D 15

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  data = (5, 10, 15, 20)
2  data = data[-3:-1]
3  data = data[-1]
4  print(data)
```

data[-3:-1] is (10, 15), a tuple. Its [-1] is the int 15.

A `(15,)`

Line 3 indexes, which gives an element, not a slice.

B `(10, 15)`

Stops after line 2. Line 3 takes one element.

C The code raises a TypeError

Nothing changes the tuple; the name is just rebound.

D **15**

The slice is (10, 15); indexing it gives the int 15.

THE SIGNAL

Slicing a tuple gives a tuple. Indexing gives an element.

What is the output of this program?

```
1  t = (1,) * 2 + (2,)
2  t += t[1:]
3  print(len(t), t)
```

A `5 (1, 1, 2, 1, 2)`

B `4 (1, 1, 2, 2)`

C `3 (1, 1, 2)`

D The code raises a TypeError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  t = (1,) * 2 + (2,)
2  t += t[1:]
3  print(len(t), t)
```

(1,) * 2 is (1, 1); plus (2,) is (1, 1, 2). t[1:] is (1, 2). Joined: (1, 1, 2, 1, 2).

A `5 (1, 1, 2, 1, 2)`
+= builds a new tuple from t and its slice.

B `4 (1, 1, 2, 2)`
Takes t[1:] as just the last element.

C `3 (1, 1, 2)`
Assumes += cannot work on a tuple and changes nothing.

D The code raises a TypeError
+= on a tuple is legal: it makes a new tuple.

THE SIGNAL

+= on a tuple is allowed. It never changes the old tuple; it builds a new one.

What is the output of this program?

```
1  t = ([0], 5)
2  t[0][0] += 1
3  u = t
4  u[0].append(9)
5  print(t)
```

A ([0], 5)

B The code raises a TypeError

C ([1, 9], 5)

D ([1], 5)

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  t = ([0], 5)
2  t[0][0] += 1
3  u = t
4  u[0].append(9)
5  print(t)
```

Both changes happen inside the list. No tuple slot is ever replaced.

A `([0], 5)`
Assumes the tuple protects the list's contents. It does not.

B The code raises a TypeError
t[0][0] changes the list, not a tuple slot. That is allowed.

C `([1, 9], 5)`
The list inside changes twice; u is the same tuple as t.

D `([1], 5)`
Forgets that u and t name the same tuple, holding the same list.

THE SIGNAL

A tuple fixes which objects it holds, not what happens inside them.

> t = (1, 2, 3) has been assigned. Each line is run on its own.

Which two lines raise an exception? (Choose two.)

A `print(t.count(4))`

B `print(t[3])`

C `print(t[1:9])`

D `t.append(4)`

◆ PAUSE AND DECIDE

Which two lines raise an exception? (Choose two.)

A `print(t.count(4))`

Counting a missing value is not an error. It prints 0.

B `print(t[3])`

Valid indexes are 0 to 2. IndexError.

C `print(t[1:9])`

Slices never raise IndexError. Prints (2, 3).

D `t.append(4)`

Tuples have no append method. AttributeError.

THE SIGNAL

count() of a missing value is 0, not an error. index() of a missing value would raise ValueError.